

Sunbeam Institute of Information Technology
Pune and Karad

Algorithms and Data structures

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

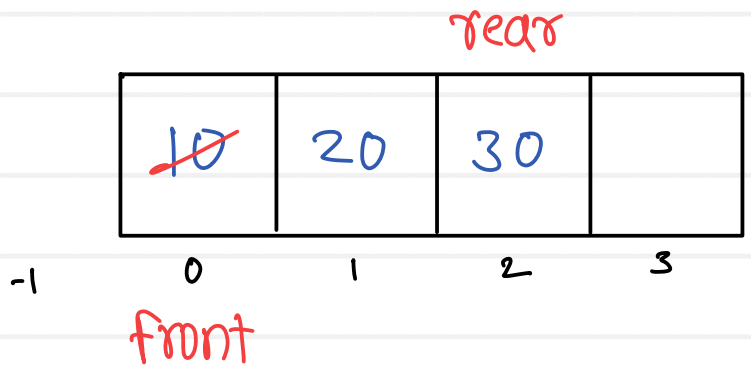
stack and queue are called as utility data structures because they are used in many applications and algorithms

- Queue
  - linear
  - Circular
  - Deque
  - Priority queue

Linear queue

- linear data structure which has two ends - front and rear
- Data is inserted from rear end and removed from front end
- Queue works on the principle of “First In First Out” / “FIFO”

Capacity = 4



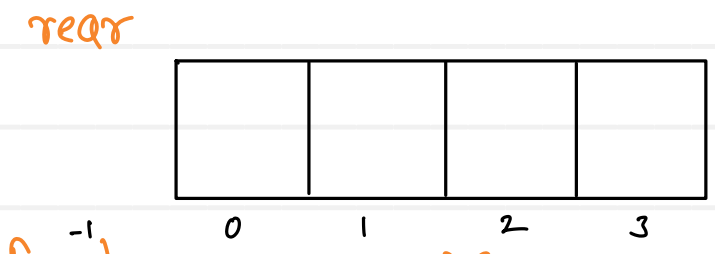
Operation :

1. Insert / Enqueue / Offer :
   i. reposition rear (inc)
   ii. insert value at rear index
2. Remove / Dequeue / poll :
   i. reposition front (inc)
3. Peek :
   i. print / return value of front end (front+1 index)

All operations of queue are performed in O(1) time complexity

Linear queue - Conditions

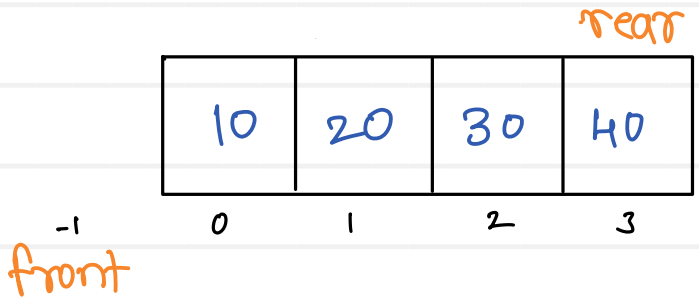
Empty



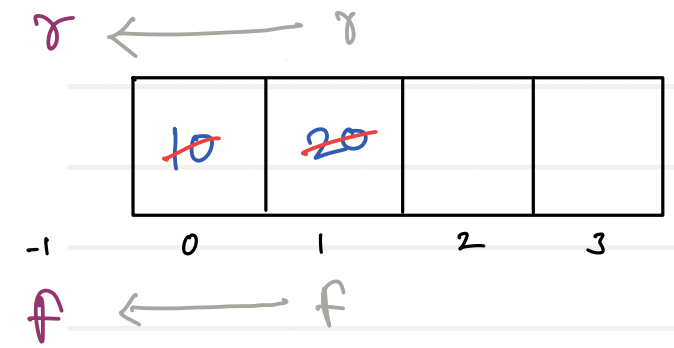
front == rear



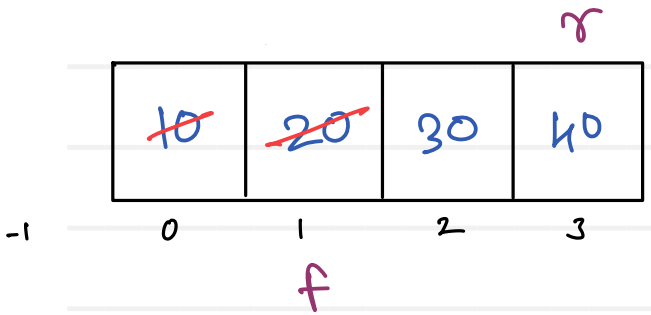
Full



rear == size-1




```
poll() :
if(front == rear)
    front = rear = -1;
```



- when queue is partially empty (initial few locations are empty), we can not reuse those locations until queue becomes empty.
- this leads to poor memory utilization

Circular queue

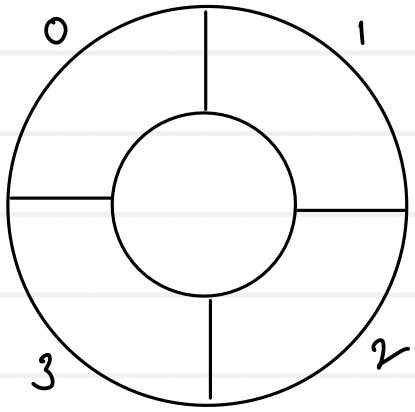
Capacity=4



operation:

1. Enqueue
   a. reposition rear (inc)
   b. add data at rear index
2. Dequeue
   a. reposition front (inc)
3. Peek
   a. read/return data of front end (front+1) index

front = (front+1) % size
rear = (rear+1) % size

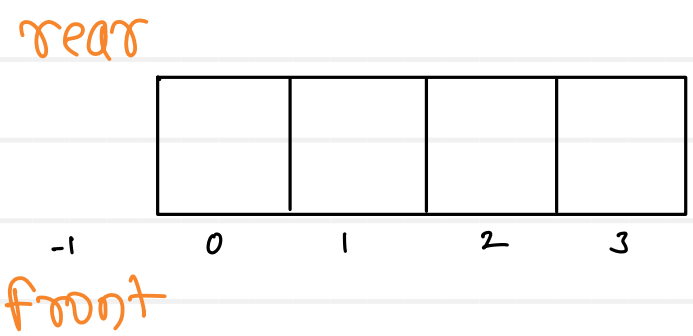


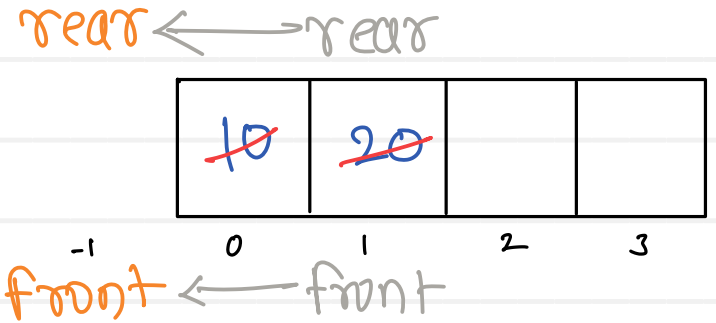
front = rear = -1
= (-1+1) % 4 = 0
= (0+1) % 4 = 1
= (1+1) % 4 = 2
= (2+1) % 4 = 3
= (3+1) % 4 = 0

- all operations of queue are performed in O(1) time complexity

Circular queue - Conditions

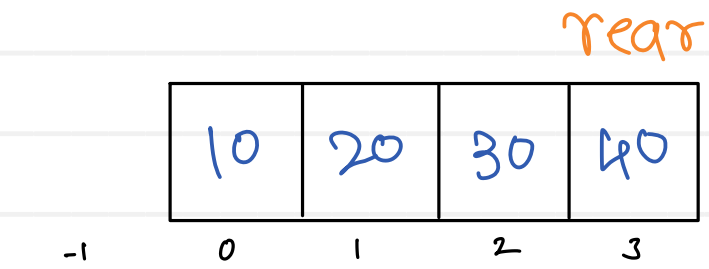
Empty

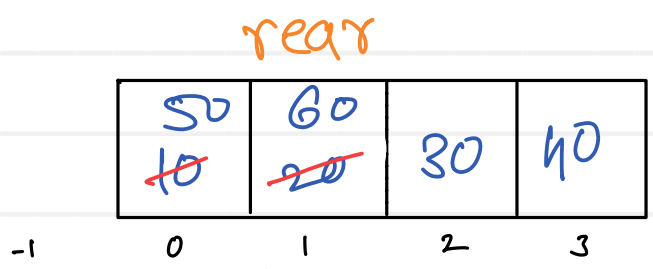




front == rear && rear == -1

Full





(front == -1 && rear == size-1) || (front == rear && rear != -1)

Applications of Queue

- job queue, ready queue or waiting queue in OS
- jobs submitted to printer [spooler directory]
- call to customer care
- token systems
- in networks to modify common files
- advanced data structures for traversal (BFS - queue)

Stack

- Stack is a linear data structure which has only one end - top
- Data is inserted and removed from top end only.
- Stack works on principle of “Last In First Out” / “LIFO”

Capacity = 4

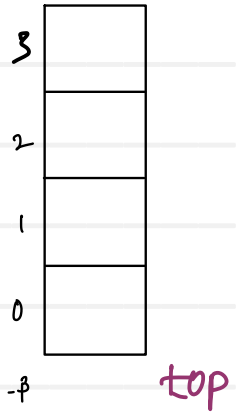


Operations :

1. Insert / Push :
   i. reposition top (inc)
   ii. add value at top index
2. Remove / Pop :
   i. reposition top (dec)
3. Peek :
   i. read value of top index

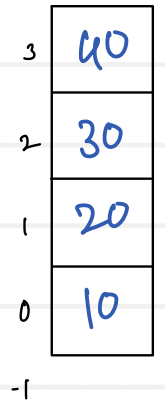
"Top always points to the last inserted data/value."

Empty



top == -1

Full



top == size-1

- all operations of stack are performed in O(1) time complexity.

Ascending stack

top = -1

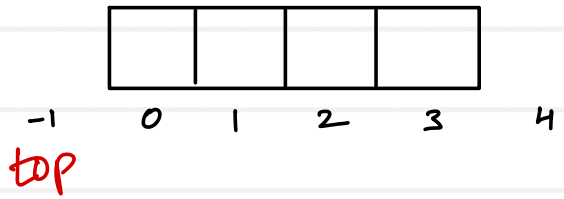
push : top++
arr[top] = value

pop : top--

peek : arr[top]

Empty : top == -1

Full : top == size-1



Descending stack

top = size

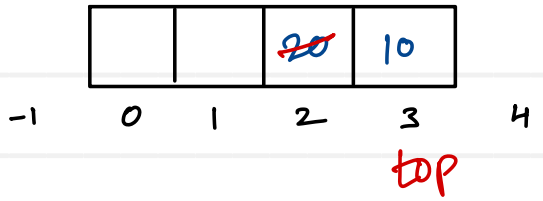
push : top--
arr[top] = value

pop : top++

peek : arr[top]

Empty : top == size

Full : top == 0



Stack Applications

- Lexical analysis in compilers
  - parenthesis balancing
- function activation records
  - ↳ control the flow program
- traversal in advanced data structures
  - DFS - stack
  - preorder/inorder/postorder - stack
- Expression conversion & evaluation

Applications – Stack and Queue

Stack

- Parenthesis balancing
- Expression conversion and evaluation
- Function calls
- Used in advanced data structures for traversing

- **Expression conversion and evaluation:**
  - Infix to postfix
  - Infix to prefix
  - Postfix evaluation
  - Prefix evaluation

Queue

- Jobs submitted to printer
- In Network setups – file access of file server machine is given to First come First serve basis
- Calls are placed on a queue when all operators are busy
- Used in advanced data structures to give efficiency.
- Process waiting queues in OS

Expression : combination of operands & operators

Types :

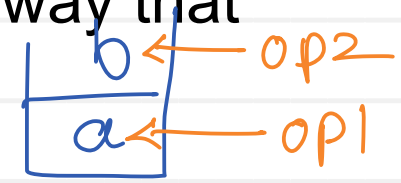
1. Infix : a + b → human
2. Prefix : + a b
3. Postfix : a b +

→ machine → CPU → ALU

Postfix Evaluation

- Process each element of postfix expression from left to right
- If element is operand
  - Push it on a stack
- If element is operator
  - Pop two elements (Operands) from stack, in such a way that
    - Op2 – first popped element
    - Op1 – second popped element
  - Perform current element (Operator) operation between Op1 and Op2
  - Again push back result onto the stack
- When single value will remain on stack, it is final result
- e.g. 4 5 6 * 3 / + 9 + 7 -

ab+



Postfix evaluation

Postfix expression : 4 5 6 * 3 / + 9 + 7 -

l ⟶ r

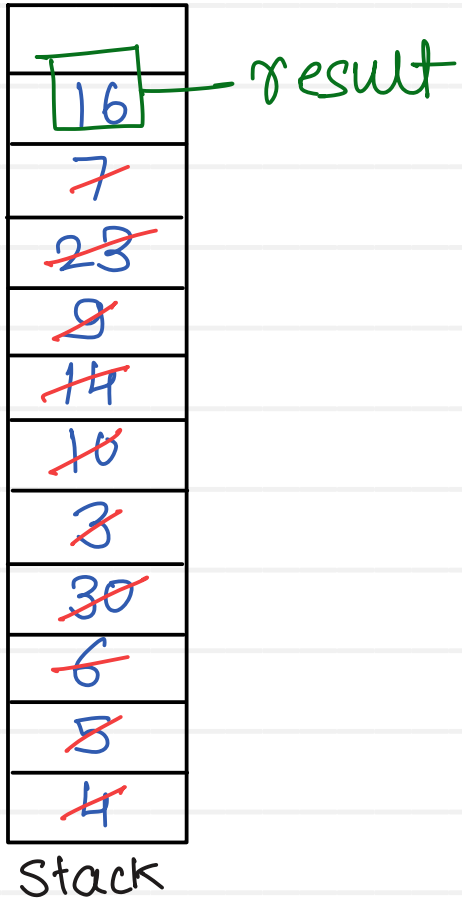
23 − 7 = 16

14 + 9 = 23

4 + 10 = 14

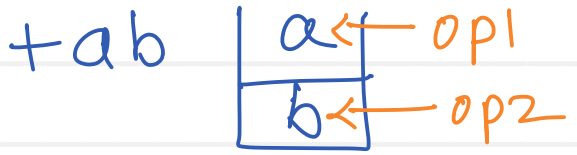
30 / 3 = 10

5 * 6 = 30



Prefix Evaluation

- Process each element of prefix expression from right to left
- If element is operand
  - Push it on a stack
- If element is operator
  - Pop two elements (Operands) from stack, in such a way that
    - Op1 – first popped element
    - Op2 – second popped element
  - Perform current element (Operator) operation between Op1 and Op2
  - Again push back result onto the stack
- When single value will remain on stack, it is final result
- e.g. - + + 4 / * 5 6 3 9 7



Prefix evaluation

Prefix expression : - + + 4 / * 5 6 3 9 7

l ← r

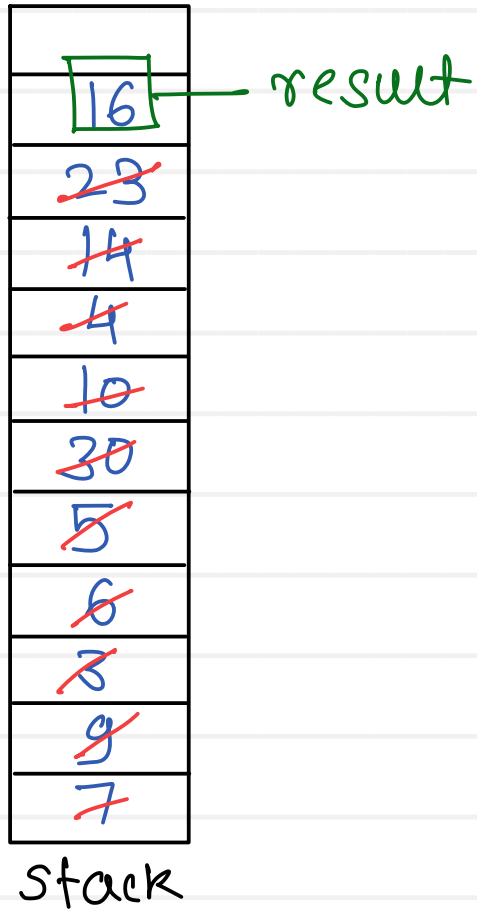
23 − 7 = 16

14 + 9 = 23

4 + 10 = 14

30 / 3 = 10

5 * 6 = 30



Infix to Postfix Conversion

- Process each element of infix expression from left to right
- If element is Operand
  - Append it to the postfix expression
- If element is Operator
  - If priority of topmost element (Operator) of stack is greater or equal to current element (Operator), pop topmost element from stack and append it to postfix expression
  - Repeat above step if required
  - Push element on stack
- Pop all remaining elements (Operators) from stack one by one and append them into the postfix expression
- e.g. a * b / c * d + e – f * h + i

Infix to Postfix conversion

Infix expression : 1 $ 9 + 3 * 4 - (6 + 8 / 2) + 7

l → r

Postfix expression : 19$34*+682/+-7+



Infix to Prefix Conversion

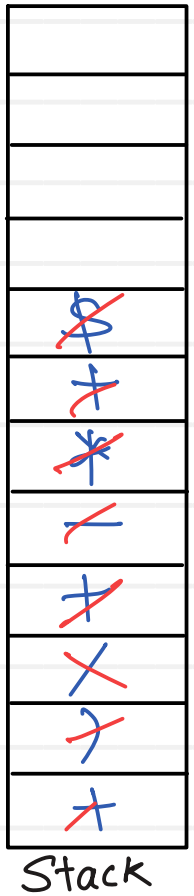
- Process each element of infix expression from right to left
- If element is Operand
  - Append it to the prefix expression
- If element is Operator
  - If priority of topmost element of stack is greater than current element (Operator), pop topmost element from stack and append it to prefix expression
  - Repeat above step if required
  - Push element on stack
- Pop all remaining elements (Operators) from stack one by one and append them into the prefix expression
- Reverse prefix expression
- e.g. a * b / c * d + e – f * h + i

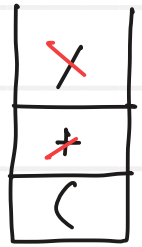
Infix to Prefix conversion

Infix expression : 1 $ 9 + 3 * 4 - (6 + 8 / 2) + 7

Expression : 728/6+43*91$+-+

Prefix expression : +-+$19*34+6/827





'/' != '('
'+' != '('
'(' != '('

while(st.peek() != '(')
append(st.pop());
st.pop();

Prefix to Postfix

- Process each element of prefix expression from right to left
- If element is an Operand
  - Push it on to the stack
- If element is an Operator
  - Pop two elements (Operands) from stack, in such a way that
    - Op1 – first popped element
    - Op2 – second popped element
  - Form a string by concatenating Op1, Op2 and Opr (element)
  - String = “Op1+Op2+Opr”, push back on to the stack
- Repeat above two steps until end of prefix expression.
- Last remaining on the stack is postfix expression
- e.g. * + a b – c d

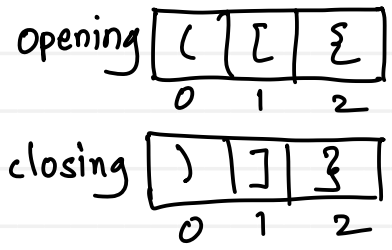
Postfix to Infix

- Process each element of postfix expression from left to right
- If element is an Operand
  - Push it on to the stack
- If element is an Operator
  - Pop two elements (Operands) from stack, in such a way that
    - Op2 – first popped element
    - Op1 – second popped element
  - Form a string by concatenating Op1, Opr (element) and Op2
  - String = “Op1+Opr+Op2”, push back on to the stack
- Repeat above two steps until end of postfix expression.
- Last remaining on the stack is infix expression
- E.g. a b c - + d e – f g – h + / *

Valid Parantheses

Given a string s containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.
An input string is valid if:
- Open brackets must be closed by the same type of brackets.
- Open brackets must be closed in the correct order.
- Every close bracket has a corresponding open bracket of the same type.

Example 1:
Input: s = "()"
Output: true
Example 2:
Input: s = "()[]{}"
Output: true
Example 3:
Input: s = "(]"
Output: false
Example 4:
Input: s = "([])"
Output: true



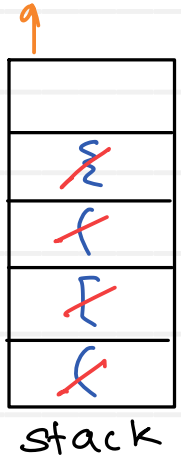
string
↓
indexOf( )
↓
returns index of char
returns -1 if char not found

1. Create stack to push brackets
2. traverse string from left to right
   2.1 if bracket is opening
       then push it on stack
   2.2 if bracket is closing
       2.2.1 if stack is empty, return false.
       2.2.2 if stack is not empty,
           - pop one bracket from stack,
           - if they are matching, continue
           - if they are not matching, return false.
3. if stack is not empty, return false
4. if stack is empty, return true

Parenthesis balancing using stack

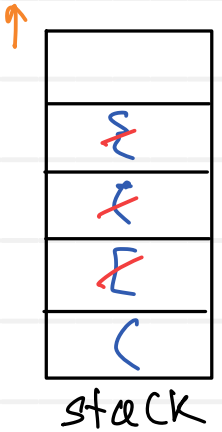
5+([9-4]*(8-{6/2}))

] == [
} == {
) == (
) == (



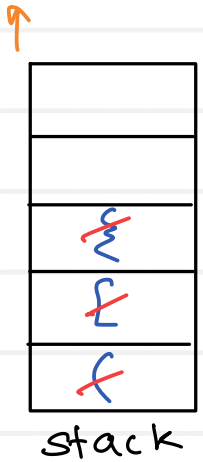
5+([9-4]*(8-{6/2])

] == [
} == {
] != (



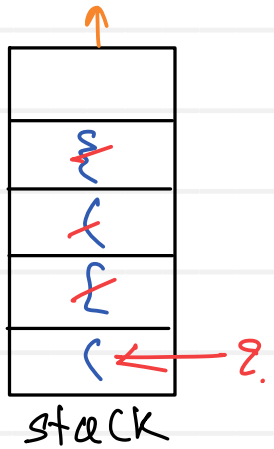
5+([9-4]*8-{6/2}))

] == [
} == {
) == (
) → 2.



5+([9-4]*(8-{6/2})

] == [
} == {
) == (



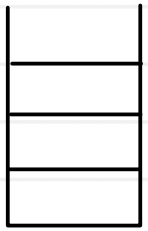
Remove all adjacent duplicates in string

You are given a string s consisting of lowercase English letters. A duplicate removal consists of choosing two adjacent and equal letters and removing them.

We repeatedly make duplicate removals on s until we no longer can.

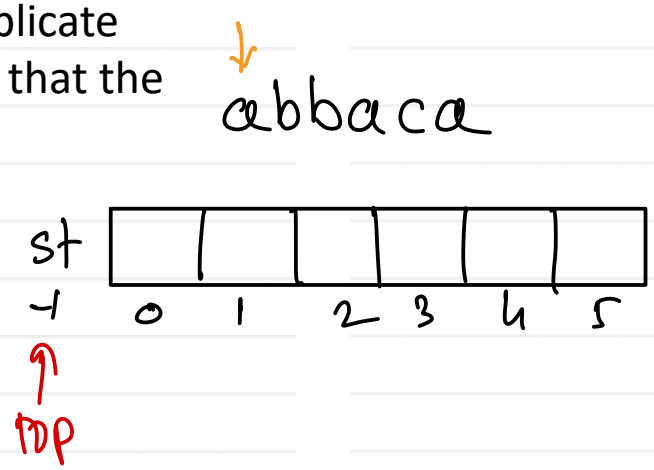
Return the final string after all such duplicate removals have been made. It can be proven that the answer is unique.

Example 1:
Input: s = "abbaca"
Output: "ca"



Example 2:
Input: s = "azxxzy"
Output: "ay"






```
string removeDuplicates(String s){
    int n = s.length();
    char st[] = new char[n];
    int top = -1;
    for(int i=0; i<n; i++){
        char curr = s.charAt(i);
if( top != -1 && curr == st[top])
        top--;
else
        st[++top] = curr;
    }
return new String(st, 0, top+1);
}
```